Guia de Sobrevivência Nível 2 (Misto)

Projeto Final DevSecOps Clube Aurora (Turma 1746)

> Bem-vindo(a) ao **Guia de Sobrevivência do Nível 2 (Misto)**!
> Este material foi elaborado para dar a você total segurança e clareza. Aqui você não precisa adivinhar nada: você terá pistas graduais, comandos validados e explicações diretas para atuar no código de 3 falhas práticas (**F1, F2, F4**) e analisar conceitualmente as outras 4 falhas (**F3, F5, F6, F7**) no seu relatório.

---

1. Visão Geral da Trilha Mista

A trilha **Nível 2 (Misto)** é o equilíbrio ideal entre engenharia prática e análise conceitual de segurança defensiva:

1.1 O que você faz no código vs. o que você analisa no relatório

Tipo de Atuação	Falhas Envolvidas	O que você fará na prática	O que você entrega
Mão na Massa (Código & Git)	**F1, F2, F4**	Você altera arquivos reais no seu fork, faz commits e observa o GitHub Actions passar de vermelho para verde nas respectivas etapas de validação.	Commits no fork + Pull Request aberto + Prints dos steps no relatório.
Análise Conceitual (Relatório)	**F3, F5, F6, F7**	Você estuda os logs e trechos de código fornecidos, diagnostica o risco e preenche as seções de modelagem de ameaças e governança.	Arquivo `TEMPLATE_TM.md` preenchido na pasta `aluno/`.

1.2 Estratégia recomendada de execução

Recomendamos que você divida seu trabalho em 4 blocos de aproximadamente 1h30:

1. **Sessão 1 (Setup e F1):** Faça o fork, ative o GitHub Actions, faça o primeiro push para ver a pipeline falhar e resolva a falha **F1** (higiene de segredos no repositório).
2. **Sessão 2 (Código F2 e F4):** Atualize as dependências Python em `requirements.txt` (**F2**) e faça o hardening do `Dockerfile` (**F4**). Suba os commits e garanta os testes verdes.
3. **Sessão 3 (Relatório Teórico Seção 1):** Preencha a matriz STRIDE e Tríade CIA para **F3, F5, F6 e F7** (utilizando o exemplo resolvido de F3 deste guia como modelo).
4. **Sessão 4 (Governança, Observabilidade e Envio):** Responda às perguntas sobre o Tio Armando (SoD e RBAC), liste os eventos de auditoria e submeta o Pull Request com seu relatório preenchido.

---

2. Metodologia de Trabalho

2.1 Como ler um erro do GitHub Actions em 5 passos

Quando o pipeline do GitHub Actions falha (ícone vermelho com `X`), não se assuste. Siga este roteiro sistemático:

```
[Aba Actions]  [Selecionar a Execução]  [Abrir o Job com Falha]  [Localizar o Step com X]  [Ler o Log do S
```

1. **Passo 1 Acesse a aba Actions:** No topo do seu repositório no GitHub, clique na aba **Actions** e selecione o workflow run que acabou de rodar (geralmente intitulado `DevSecOps CI/CD Pipeline`).
2. **Passo 2 Identifique o Job afetado:** A pipeline possui dois jobs: `pipeline-estatica` (SAST, SCA e Linter) e `pipeline-dinamica` (Container Security e DAST). Veja qual deles tem um círculo vermelho.
3. **Passo 3 Expanda o Step com falha:** Clique no step que contém o ícone vermelho (por exemplo, `Análise de Dependências (Trivy FS)` ou `Análise de Dockerfile (Hadolint)`).
4. **Passo 4 Localize a tabela ou código de erro:**
   - No **Trivy**, procure a tabela que lista colunas como `Library`, `Vulnerability`, `Severity` e `Installed Version`.
   - No **Hadolint**, procure por códigos iniciados em `DL` (ex: `DL3007`).
   - No **Semgrep**, procure pelo identificador da regra e número da linha do arquivo.
5. **Passo 5 Identifique a ação requerida:** Anote qual pacote precisa ser atualizado ou qual instrução do arquivo precisa de ajuste antes de mexer no código.

2.2 Como fazer commits pequenos e atômicos

Evite fazer todas as alterações de uma vez só. No DevSecOps profissional, cada correção gera um commit isolado para que o time de segurança possa auditar o impacto:

- **1 falha resolvida = 1 commit = 1 disparo de pipeline.**
- Antes de commitar, sempre verifique exatamente o que você alterou no seu terminal: `bash git status git diff`
- Use mensagens claras seguindo o padrão Conventional Commits:
- `fix(security): sanitize environment variables and update gitignore` (para F1)
- `fix(deps): update vulnerable python packages in requirements.txt` (para F2)
- `fix(docker): harden Dockerfile with nonroot user and healthcheck` (para F4)
- `docs(relatorio): complete theoretical threat analysis and governance` (para o relatório)

2.3 O que fazer se você travar (Os 3 níveis de socorro)

Se você ficar em dúvida ou o pipeline continuar vermelho após uma alteração:

1. **Nível 1 (Autonomia Guiada):** Abra as caixas de **Pistas Progressivas** na seção 3 deste guia. Elas foram construídas para orientar o seu raciocínio sem entregar a resposta imediatamente.
2. **Nível 2 (Colaboração Ética):** Chame um colega da turma ou use o canal de dúvidas da Ada. Descreva o sintoma: *"Estou com o erro DL3007 no Hadolint, entendi que é sobre tag mutável, mas não sei qual versão usar"*. Jamais envie ou copie arquivos prontos.
3. **Nível 3 (Mentoria com a Professora):** Se persistir por mais de 30 minutos, procure a **professora** na aula ou nos canais oficiais da disciplina com:
   - O link do seu repositório ou Pull Request;
   - O print ou trecho do log do GitHub Actions onde ocorreu a falha;
   - O que você já tentou alterar.

2.4 Como validar localmente antes do push (Script de Self-Check)

Para não precisar esperar a fila do GitHub Actions rodar a cada pequena alteração, você pode testar suas correções diretamente no seu terminal com o validador local de autoavaliação (usa apenas biblioteca padrão do Python, sem depender de Docker):

```
python scripts/selfcheck.py --nivel 2
```

- Se houver pendências em F1, F2 ou F4, o script exibirá um marcador `[ ]` com uma dica progressiva de correção e sairá com código de erro (`exit code 1`).
- Quando as 3 falhas estiverem sanadas, o script exibirá `[OK]` em todas elas e sairá com código de sucesso (`exit code 0`), indicando que você está pronto para commitar e abrir o Pull Request!

---

3. Falhas Corrigidas no Código (F1, F2, F4)

Aqui estão as 3 falhas que você corrigirá no código-fonte do seu repositório.

F1 Segredos expostos em `.env` e ausência no `.gitignore`

O que a ferramenta mostrou (e por que olhar o repositório)

O scanner padrão de segredos do Trivy hoje analisa padrões específicos de chaves de nuvem e certificados. Por esse motivo, senhas locais de desenvolvimento em arquivos `.env` muitas vezes **não são bloqueadas automaticamente** pela ferramenta sem regras customizadas.
No entanto, na inspeção de boas práticas e auditoria do repositório, salta aos olhos:
- O arquivo `.env` está versionado no Git com credenciais sensíveis reais:
- `SECRET_KEY="aurora-dev-secret-change-me"`
- `SEED_PASSWORD="aurora-seed-password-2024"`
- `ADMIN_INITIAL_TOKEN="aurora-admin-token-xyz987"`
- O arquivo `.gitignore` na raiz do projeto existe, mas **não contém** a linha `.env`. Qualquer desenvolvedor que clonar o projeto continuará commitando segredos para a nuvem.

Onde olhar no repositório

- Arquivo `.env` (na raiz do repositório)
- Arquivo `.gitignore` (na raiz do repositório)

[NOTA] Pista 1 (Conceito de higiene de credenciais) Arquivos `.env` contêm segredos específicos de cada ambiente (desenvolvimento, homologação, produção). Eles **nunca** devem ser salvos no histórico do Git, pois qualquer pessoa com acesso de leitura ao repositório terá posse dessas credenciais. O padrão da indústria é manter no repositório apenas um arquivo modelo (ex: `.env.example`) contendo as variáveis vazias ou com valores fictícios genéricos, enquanto o arquivo `.env` real fica protegido no disco local e listado no `.gitignore`. [ANÁLISE] Pista 2 (Direção da correção prática) Você precisa realizar 3 ações: 1. Abrir `.gitignore` e adicionar a linha `.env`. 2. Criar uma cópia do `.env` chamada `.env.example`, substituindo os segredos reais por instruções claras (ex: `SECRET_KEY="altere-para-uma-chave-forte-em-producao"`). 3. Remover o arquivo `.env` do índice do Git para que ele pare de ser rastreado, mas sem apagá-lo da sua máquina local: ```bash git rm --cached .env ``` [DAST] Estratégia de Resolução (Passo a Passo Conceitual) 1. **Atualize o `.gitignore`:** Adicione uma linha dedicada para que arquivos `.env` não sejam mais incluídos em novos commits. 2. **Crie o `.env.example`:** Crie um arquivo com a mesma estrutura do seu `.env`, porém com valores fictícios de modelo (ex: `"altere-em-producao"`). 3. **Desindexe o `.env` do Git:** Use o comando `git rm --cached .env`

para removê-lo da área de rastreamento do repositório sem deletar o arquivo do seu disco local. 4. **Validação e Dicas de Debugging:** Se tiver dúvidas de comando ou enfrentar problemas ao testar, consulte o roteiro interativo em [`aluno/PLANO_DE_TESTES_DEBUGGING.md`](./PLANO_DE_TESTES_DEBUGGING.md#2-etapa-1--higiene-de-reposit

Como provar que corrigiu

- Execute no seu terminal:
  `bash git ls-files .env`
  O comando **não deve retornar nada** (saída vazia), provando que o Git não rastreia mais o arquivo sensível.
- Verifique se `.env.example` aparece listado em `git ls-files .env.example`.

F2 Dependências Python vulneráveis (`requirements.txt`)

O que a ferramenta mostrou (excerto real do Trivy FS)

No log da **Pipeline Estática**, o step `Análise de Dependências (Trivy FS)` falha com saída similar a:

```
Total: 10 (HIGH: 10, CRITICAL: 0)


 Library   Vulnerability   Severity  Status  Installed Version  Fixed Version  Title

 gunicorn  CVE-2024-1135   HIGH      fixed   20.1.0             23.0.0         HTTP request/response smugg
 gunicorn  CVE-2024-6827   HIGH      fixed   20.1.0             23.0.0         HTTP request/response smugg
 urllib3   CVE-2023-43804  HIGH      fixed   1.26.5             1.26.17        Cookie leak on cross-origin
 urllib3   CVE-2025-66418  HIGH      fixed   1.26.5             1.26.20        Streaming body reading vuln
 urllib3   CVE-2025-66471  HIGH      fixed   1.26.5             1.26.21        HTTP Header Injection
 urllib3   CVE-2026-21441  HIGH      fixed   1.26.5             2.2.2          Decompression bomb handling
 urllib3   CVE-2026-44431  HIGH      fixed   1.26.5             2.2.2          Proxy authentication header
 urllib3   CVE-2026-97687  HIGH      fixed   1.26.5             2.2.2          Connection pool exhaustion
 urllib3   CVE-2026-97689  HIGH      fixed   1.26.5             2.2.2          TLS session reuse flaw
 werkzeug  CVE-2024-34069  HIGH      fixed   2.2.3              3.0.3          Routing parse crash / ReDoS


```

Onde olhar no repositório

- Arquivo `requirements.txt` (na raiz do repositório):
  `text flask==2.2.5 werkzeug==2.2.3 gunicorn==20.1.0 urllib3==1.26.5`

[NOTA] Pista 1 (Conceito de SCA e compatibilidade) O **Software Composition Analysis (SCA)** identifica bibliotecas de terceiros que possuem CVEs registradas em bases públicas. O Trivy compara as versões declaradas no seu arquivo de manifesto com o banco do NVD/GitHub Advisory. Repare que o Flask depende do Werkzeug. Ao atualizar o Werkzeug para `>=3.0.3`, o Flask antigo (`2.2.5`) pode ter incompatibilidades. Portanto, a prática correta é atualizar ambos para suas versões estáveis modernas. [ANÁLISE] Pista 2 (Direção da correção das versões) Olhe a coluna `Fixed Version` da tabela acima: - `gunicorn` precisa ir para `23.0.0`; - `urllib3` precisa ir para uma versão superior a `2.2.2` (ex: `2.8.0`); - `werkzeug` precisa ir para `>=3.0.3`; - `flask` deve acompanhar para `3.0.3` para manter total consistência com o Werkzeug. [DAST] Estratégia de Resolução (Passo a Passo Conceitual) 1. **Consulte a coluna `Fixed Version` do Trivy:** Identifique no log do scanner a versão recomendada para cada pacote com CVE (Gunicorn, Werkzeug, urllib3). 2. **Alinhe os pacotes dependentes:** Ao promover o Werkzeug para a série 3.x, atualize também o Flask para a série 3.x para evitar conflitos de imports descontinuados. 3. **Edite o `requirements.txt`:** Fixe as versões estáveis identificadas. 4. **Validação e Dicas de Debugging:** Para entender erros comuns de importação e comandos de teste locais, consulte o roteiro interativo em [`aluno/PLANO_DE_TESTES_DEBUGGING.md`](./PLANO_DE_TESTES_DEBUGGING.md#3-etapa-2--sca--compatibilida

Como provar que corrigiu

- Acesse a aba **Actions** após o push. O step `Análise de Dependências (Trivy FS)` executará com sucesso e relatará:
  `Total: 0 (HIGH: 0, CRITICAL: 0)`

F4 Configuração insegura de contêiner (`Dockerfile`)

O que a ferramenta mostrou (excerto real do Hadolint)

Na **Pipeline Estática**, o linter de Dockerfile (`Hadolint`) reporta a seguinte violação na linha 6:


```
[
  {
    "line": 6,
    "code": "DL3007",
    "message": "Using latest is prone to errors if the image will pull a higher version than expected - us
    "column": 1,
    "file": "Dockerfile",
    "level": "warning"
  }
]
```


Além disso, ao inspecionar o `Dockerfile` original, notamos violações graves dos **4 Mandamentos do Container Hardening**:
1. Uso de tag mutável `:latest` (`FROM python:latest`);
2. Ausência de usuário não-root (o contêiner executa como `root` por padrão);
3. Ausência de instrução `HEALTHCHECK` (o orquestrador não sabe se o serviço travou);
4. Ausência de build em múltiplos estágios (deixando ferramentas de compilação dentro da imagem final de produção).

Onde olhar no repositório

- Arquivo `Dockerfile` (na raiz do repositório)

[NOTA] Pista 1 (Conceito de imagem base, privilégios e saúde) - **Tag Imutável e Minimalista:** Usar `python:3.11-slim` fixa a versão da linguagem e reduz drasticamente o tamanho da imagem, eliminando centenas de pacotes do sistema operacional que você não usa. - **Usuário sem privilégios:** Se um invasor explorar uma falha na sua aplicação Flask, ele obterá apenas as permissões do usuário sob o qual o processo roda. Se rodar como `root`, ele terá controle total dentro do contêiner. - **Healthcheck:** Permite que o Docker verifique periodicamente se a rota `/health` responde com código HTTP 200. [ANÁLISE] Pista 2 (Direção da correção prática) Você pode adotar o padrão **Multi-stage Build**: - **Estágio 1 (Builder):** Baixa o `python:3.11-slim`, instala as bibliotecas via `pip` em um diretório temporário (`/root/.local`). - **Estágio 2 (Runtime Final):** Começa com uma imagem limpa `python:3.11-slim`, cria um usuário sem privilégios (`nonroot`), copia apenas os arquivos já compilados do estágio anterior, declara o `HEALTHCHECK` e define `USER nonroot`. [DAST] Estratégia de Resolução (Passo a Passo Conceitual) 1. **Adote Multi-Stage Build:** Separe o Dockerfile em dois estágios (`AS builder` para compilar e baixar pacotes com `pip install --user`, e o estágio final enxuto para execução de runtime). 2. **Fixe a Imagem Base:** Substitua `:latest` por uma versão mínima imutável (ex: `python:3.11-slim`). 3. **Crie e Ative Usuário Não-Root:** Adicione um usuário sem privilégios (`useradd -u 10001 -m nonroot`), configure as permissões das pastas da aplicação e de dados do SQLite (`chown -R nonroot:nonroot`) e declare `USER nonroot`. 4. **Adicione HEALTHCHECK:** Configure a instrução `HEALTHCHECK` consultando a rota `/health`. 5. **Validação e Dicas de Debugging:** Se enfrentar

erros de permissão do SQLite ou dúvidas sobre comandos de compilação, consulte o roteiro interativo em [`aluno/PLANO_DE_TESTES_DEBUGGING.md`](./PLANO_DE_TESTES_DEBUGGING.md#4-etapa-3--linter--hardening-d

Como provar que corrigiu

- O step `Análise de Dockerfile (Hadolint)` na Pipeline Estática passará com status verde (sem erros `DL3007`).
- Quando você rodar a Pipeline Dinâmica, o step `Trivy Container Image Scan` encontrará zero vulnerabilidades críticas na nova imagem construída!

---

4. Falhas Analisadas no Relatório (F3, F5, F6, F7)

No **Nível 2**, você não precisa codificar a solução destas 4 falhas no seu repositório. O seu papel é atuar como **Analista de Segurança / Engenheiro(a) Consultor(a)**, documentando a causa-raiz, os impactos no negócio e a recomendação técnica no arquivo `TEMPLATE_TM.md`.

F3 Injeção de SQL via interpolação de strings (`app/db.py`)

O que a ferramenta mostrou (excerto real Semgrep)

No log da **Pipeline Estática**, o Semgrep acusou duas regras no arquivo `app/db.py` linha 102:

```
2 Code Findings

app/db.py:102
  rule: python.lang.security.audit.formatted-sql-query.formatted-sql-query
  rule: python.sqlalchemy.security.audit.sqlalchemy-execute-raw-query...

100    query = f"SELECT * FROM clients WHERE full_name LIKE '%{name}%' ORDER BY id"
101
102    rows = get_db().execute(query).fetchall()
```

Onde olhar no código-fonte

- Arquivo `aurora/app/db.py`, função `search_clients(name)` (linhas 99 a 104).

Diagnóstico Técnico & Causa-Raiz

A instrução utiliza interpolação direta de strings (`f"..."`) com a variável de entrada `name` vinda da requisição web. Como não há validação ou escape, um invasor pode passar parâmetros como `' OR '1'='1` ou `' UNION SELECT ...`, manipulando a sintaxe da consulta SQLite executada pelo banco de dados.

Proposta de Mitigação Defensiva

Substituir a concatenação/formatação de strings por **Consultas Parametrizadas (Prepared Statements)** nativas do DB-API do Python / SQLite, utilizando o marcador de posição `?`:

```
def search_clients(name):
    # Correção: O parâmetro é tratado como dado puro, nunca como instrução SQL
    rows = get_db().execute(
        "SELECT * FROM clients WHERE full_name LIKE ? ORDER BY id",
        (f"%{name}%",),
    ).fetchall()
    return [row_to_dict(r) for r in rows]
```

F5 Ausência de cabeçalhos HTTP defensivos

O que o ZAP verifica (e por que não executou no início)

O scanner de segurança dinâmico (**OWASP ZAP**) realiza requisições HTTP contra a aplicação em execução e inspeciona os cabeçalhos de resposta. No projeto vulnerável original, a **Pipeline Dinâmica falha antes no Trivy Image** (com código de saída 1 devido aos pacotes vulneráveis). Como o step de inicialização da aplicação não possui a diretiva `if: always()`, ele é **ignorado**, e o ZAP sequer chega a ser acionado no baseline inicial!

Quando a aplicação é iniciada e o ZAP executa, ele detecta a ausência de 4 cabeçalhos essenciais:
1. `Content-Security-Policy (CSP)` (ZAP Rule 10038): Previne execução de scripts não autorizados (XSS).
2. `X-Frame-Options` ou CSP `frame-ancestors` (ZAP Rule 10020): Previne ataques de Clickjacking.
3. `X-Content-Type-Options: nosniff` (ZAP Rule 10021): Impede que o navegador infira tipos MIME diferentes dos declarados (MIME sniffing).
4. `Strict-Transport-Security (HSTS)` (ZAP Rule 10035): Força conexões HTTPS criptografadas.

Onde a proteção deveria residir

A aplicação Flask em `app/__init__.py` ou `app/main.py` não registra nenhum hook de resposta nem middleware de cabeçalhos.

Proposta de Mitigação Defensiva

Pode ser resolvido no Flask através de um hook `@app.after_request`:

```
@app.after_request
def set_security_headers(response):
    response.headers["X-Content-Type-Options"] = "nosniff"
    response.headers["X-Frame-Options"] = "DENY"
    response.headers["Content-Security-Policy"] = "default-src 'self'"
    response.headers["Strict-Transport-Security"] = "max-age=31536000; includeSubDomains"
    return response
```

Ou alternativamente através da biblioteca `flask-talisman`.

F6 Quebra de privilégio mínimo e autorização frouxa (`app/db.py` e `app/main.py`)

O que a revisão manual de código descobre

Ferramentas SAST automáticas frequentemente não entendem lógica de negócio e regras de controle de acesso (Broken Access Control - OWASP Top 1). Na revisão do Clube Aurora, encontramos dois problemas graves:
1. **Em `aurora/app/db.py` (linha 58):** A função de criação de usuários define como padrão `role="admin"`: `python def create_user(username, password_hash, role="admin"):`
Qualquer novo usuário cadastrado na plataforma ganha privilégios de administrador automaticamente.
2. **Em `aurora/app/main.py` (linhas 92 a 94):** A rota de alteração de papéis `/admin/users/<int:user_id>/role` exige apenas autenticação comum: `python @bp.post("/admin/users/<int:user_id>/role") @auth.login_required def admin_set_role(user_id):`
Qualquer usuário comum autenticado (ex: cliente ou recepcionista) pode enviar uma requisição `POST` e promover qualquer conta para `admin`!

Diagnóstico Técnico

Violação direta do princípio de **Menor Privilégio (Least Privilege)** e **Controle de Acesso Quebrado**. Há elevação de privilégio horizontal e vertical sem verificação de autorização (RBAC).

Proposta de Mitigação Defensiva

1. Em `app/db.py`, alterar o valor padrão do argumento para `role="client"`.
2. Em `app/main.py`, adicionar a trava de autorização `@auth.require_role("admin")` logo abaixo do decorador de rota `@bp.post(...)`.

F7 Permissões excessivas em contêineres (`docker-compose.yml`)

O que o arquivo compose declara indevidamente

No arquivo `aurora/docker-compose.yml`, encontramos configurações de alto risco operacional:
- Linha 7: `privileged: true` (executa o contêiner com todos os recursos e capacidades do kernel Linux do host);
- Linha 10: `ports: - "5000:5000"` (exposição de porta de desenvolvimento);
- Linha 12: `SECRET_KEY: "compose-insecure-secret"` (chave secreta hardcoded);
- Linha 15: `volumes: - /var/run/docker.sock:/var/run/docker.sock` (montagem do socket de controle do Docker dentro do contêiner da aplicação).

Diagnóstico Técnico & Risco de Escape (Container Breakout)

A combinação de `privileged: true` com a montagem de `/var/run/docker.sock` permite que qualquer invasor que obtenha execução remota de código (RCE) na aplicação web converse diretamente com o daemon do Docker no servidor hospedeiro. A partir daí, o invasor pode criar um novo contêiner montando o disco raiz do servidor (`/`) e obter **acesso total como root à máquina host** (Container Breakout total).

Proposta de Mitigação Defensiva

- Remover `privileged: true`;
- Remover o bind mount de `/var/run/docker.sock`;
- Mapear apenas a porta da aplicação (`8080:8080`);
- Utilizar arquivos de ambiente externos ou secrets managers para fornecer chaves secretas.

---

5. Modelagem STRIDE e Impacto CIA

No arquivo `aluno/TEMPLATE_TM.md`, você deve preencher a tabela de modelagem de ameaças. Para que você tenha um referencial de padrão ouro, **resolvemos a falha F3 integralmente abaixo**.

5.1 Exemplo resolvido: F3 (Injeção de SQL)

Veja exatamente como uma linha de análise de ameaças deve ser construída:

Coluna no Template	Preenchimento Modelo para F3	Explicação do Raciocínio
Achado	**F3: SQL Injection**	Identificador canônico da falha no projeto.
Componente / Arquivo	`app/db.py` (`search_clients`)	Localização exata da função afetada no código.

Coluna no Template	Preenchimento Modelo para F3	Explicação do Raciocínio
Categoria STRIDE	**Tampering** (Adulteração de Dados) & **Information Disclosure** (Divulgação de Informação)	O atacante pode alterar a sintaxe SQL para ler dados de clientes não autorizados (Information Disclosure) ou modificar/excluir registros caso o driver suporte comandos múltiplos (Tampering).
Pilar CIA Afetado	**Confidencialidade** (Primário) e **Integridade** (Secundário)	A confidencialidade dos dados cadastrais dos clientes de alta renda do Clube Aurora é totalmente violada. Se houver injeção destrutiva, a integridade da base também é corrompida.
Impacto no Negócio do Clube Aurora	Vazamento massivo de PII (Dados Pessoais Identificáveis) dos membros VIP do clube, acarretando pesadas multas da LGPD (ANPD), quebra irremediável da reputação de discrição e confidencialidade da marca, e potencial chantagem contra associados.	Tradução do risco técnico para as consequências financeiras, legais e de imagem da empresa.

5.2 Guia para preenchimento dos demais achados no `TEMPLATE_TM.md`

Ao preencher as linhas de **F5, F6 e F7**, siga as seguintes diretrizes:

- **F5 (Ausência de Headers HTTP):**
- *STRIDE:* Pense em **Tampering** (injeção de scripts maliciosos em XSS devido à falta de CSP) e **Elevation of Privilege** (Clickjacking induzindo o usuário a clicar em botões administrativos ocultos).
- *Pilar CIA:* **Integridade** e **Confidencialidade**.
- *Impacto no Negócio:* Clientes ou atendentes tendo suas sessões sequestradas por scripts injetados em navegadores desatualizados.
- **F6 (Quebra de Menor Privilégio):**
- *STRIDE:* Categoria óbvia: **Elevation of Privilege** (Elevação de Privilégio horizontal e vertical).
- *Pilar CIA:* **Integridade** e **Confidencialidade** (um usuário comum ganha poder de alterar dados de qualquer reserva).
- *Impacto no Negócio:* Fraudes internas, cancelamento malicioso de reservas de terceiros e vazamento de histórico financeiro.
- **F7 (IaC Permissiva no Docker Compose):**
- *STRIDE:* **Elevation of Privilege** (Escape do contêiner para o host) e **Denial of Service** (capacidade de desligar contêineres vizinhos no daemon).
- *Pilar CIA:* Afeta os 3 pilares simultaneamente (**Confidencialidade, Integridade e Disponibilidade**).
- *Impacto no Negócio:* Comprometimento total da infraestrutura de servidores do Clube Aurora na nuvem, permitindo que invasores instalem ransomware ou utilizem as máquinas para ataques externos.

---

6. Checklist de Validação Local e Remota

Antes de considerar o seu Nível 2 concluído, passe por este checklist:

Antes de dar o push final:

- [ ] O arquivo `.env` não é mais rastreado pelo Git (`git ls-files .env` retorna vazio).
- [ ] O arquivo `.gitignore` possui a linha `.env` .
- [ ] O arquivo `.env.example` foi criado com valores fictícios.
- [ ] O arquivo `requirements.txt` foi atualizado com versões seguras de Flask, Werkzeug, Gunicorn e Urllib3.
- [ ] O arquivo `Dockerfile` utiliza imagem específica, usuário não-root e healthcheck.
- [ ] O arquivo `aluno/TEMPLATE_TM.md` está preenchido com suas respostas completas para as seções 1, 2 e 3.

No GitHub Actions (após o push):

- [ ] O job `pipeline-estatica` executou com sucesso (verde) nos passos de Hadolint e Trivy FS.
- [ ] O Pull Request para a branch `main` do seu fork foi aberto com título descritivo e relatório anexado.
- [ ] Você tirou prints dos passos verdes do Actions para incorporar nas suas evidências de entrega!